

Laporan Praktikum Pemrograman Berorientasi Objek

Pekan 2

Membuat Fungsi CRUD (Create, Read, Update, Delete) User Dengan Database MySql



Oleh : Iqbal Miftahul Fikri

NIM 2411531004

Dosen Pengampu : Nurfiah, S.ST, M.Kom.

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

Padang

2025

A. Pendahuluan

MySQL adalah sebuah sistem manajemen basis data relasional (RDBMS) open-source yang banyak digunakan untuk mengelola data dalam sebuah aplikasi. Dengan MySQL, data dapat disimpan, diatur, dan diakses dalam bentuk tabel. MySQL juga dapat diintegrasikan dengan aplikasi berbasis Java melalui MySQL Connector, sehingga memungkinkan aplikasi tersebut untuk melakukan berbagai operasi seperti menyimpan, mengubah, mengambil, serta menghapus data secara langsung. Fungsi utama dari MySQL Connector meliputi membuka koneksi ke database MySQL, mengirimkan perintah SQL ke server, menerima hasil dari perintah tersebut, dan menutup koneksi setelah proses selesai.

B. Tujuan

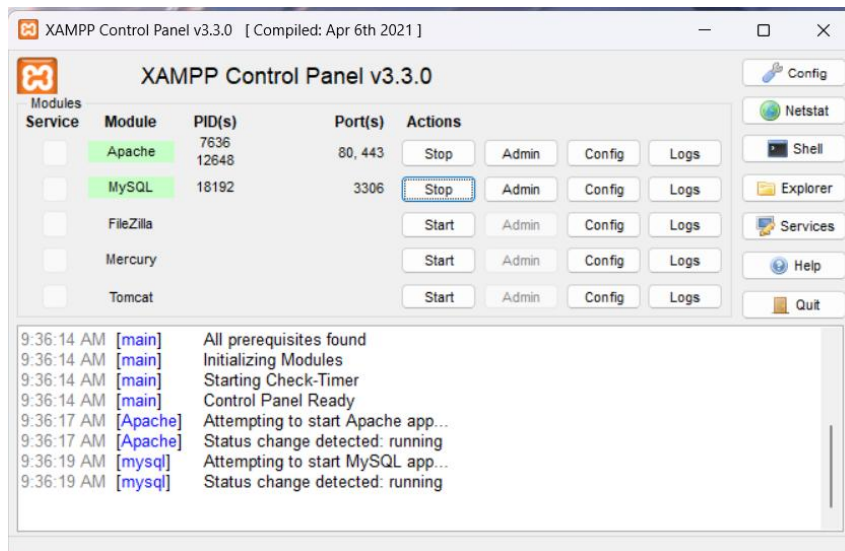
Berikut tujuan dari praktikum ini :

1. Mahasiswa dapat membuat tabel user di dalam database MySQL
2. Mahasiswa dapat menghubungkan aplikasi Java dengan database MySQL
3. Mahasiswa dapat merancang tampilan antarmuka pengguna (GUI) untuk operasi CRUD pada user
4. Mahasiswa dapat membuat dan menerapkan interface dalam pemrograman

C. Langkah – Langkah

A. Install XAMPP

1. Pertama-tama Download dan XAMPP terlebih dahulu
2. Setelah selesai, jalankan XAMPP dan aktifkan apache dan mysql

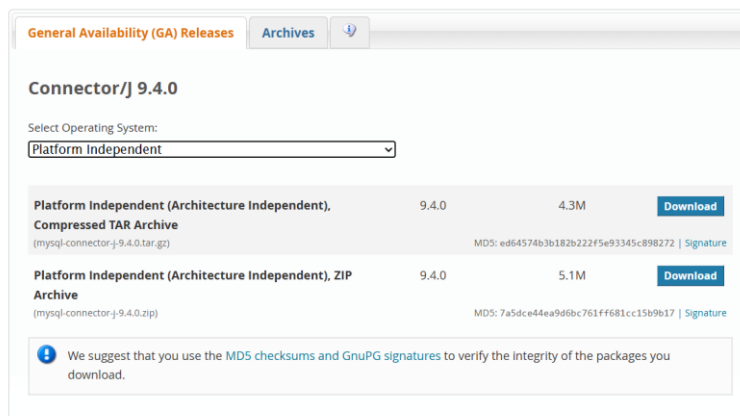


B. Menambahkan MySQL Connector

1. Download terlebih dahulu MySQL Connection pada link ini <https://dev.mysql.com/downloads/connector/j/>
2. Setelah itu download file yang berekstensi zip

MySQL Community Downloads

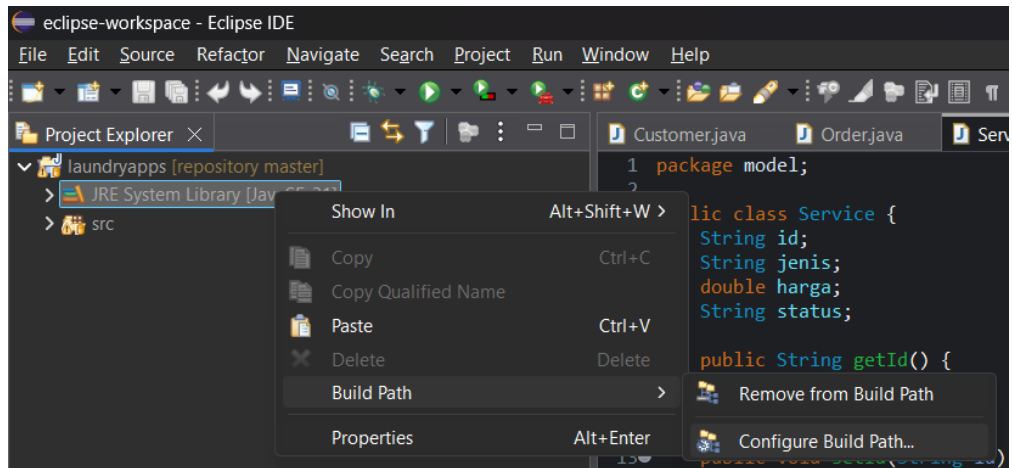
Connector/J



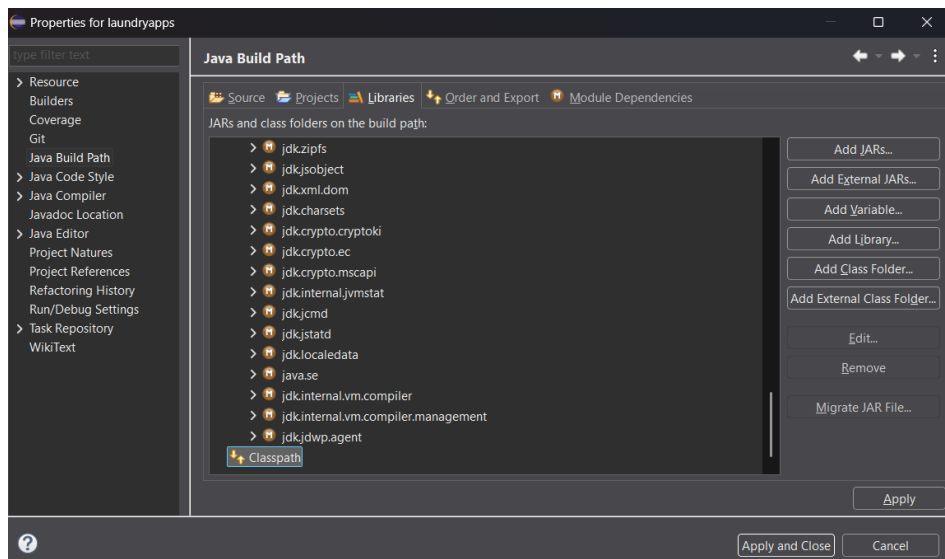
ORACLE © 2025 Oracle

Privacy / Do Not Sell My Info | Terms of Use | Trademark Policy | Cookie Preferences

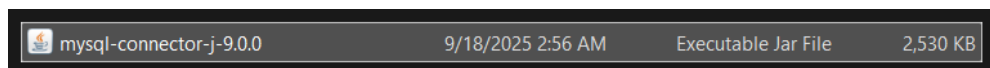
3. Setelah selesai di download, tambahkan MySQL Connector kedalam project dengan cara, klik kanan pada JRE System Library → Build Path → Configure Build Path...



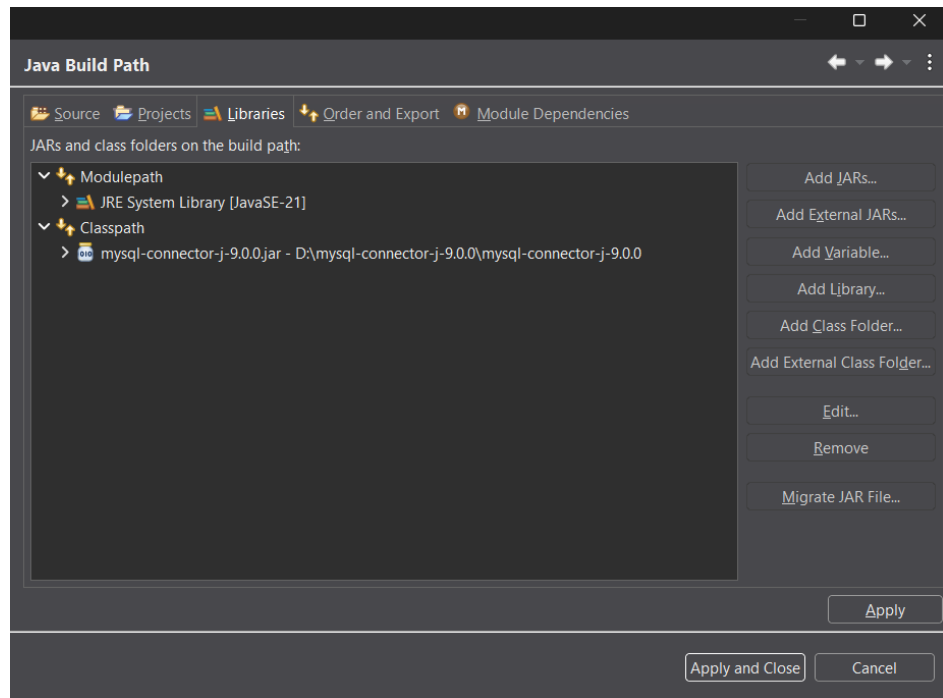
4. Lalu pilih Libraries → Classpath



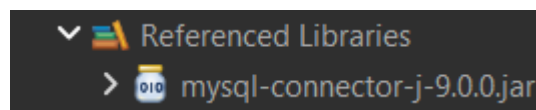
5. Sebelum kita klik Add External JARs, ekstrak terlebih dahulu file MySQL Connector yang telah di download tadi agar bertipe Jar



6. Jika sudah, klik Add External JARs, lalu pilih file yang sudah di ekstrak tadi, lalu klik Apply and Close



7. Jika berhasil maka akan generate folder Referenced Libraries pada project yang berisi MySQL Connector



C. Membuat Database dan Table User

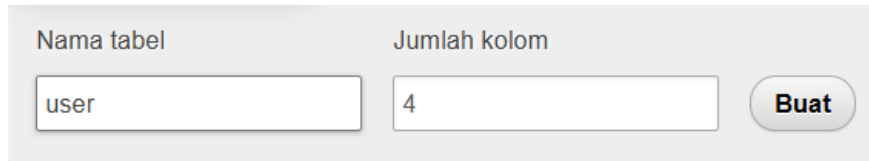
1. Buka phpmyadmin dengan mengklik link ini <http://localhost/phpmyadmin>
2. Klik new dan buat database dengan nama laundry_apps

Basis data

Buat basis data

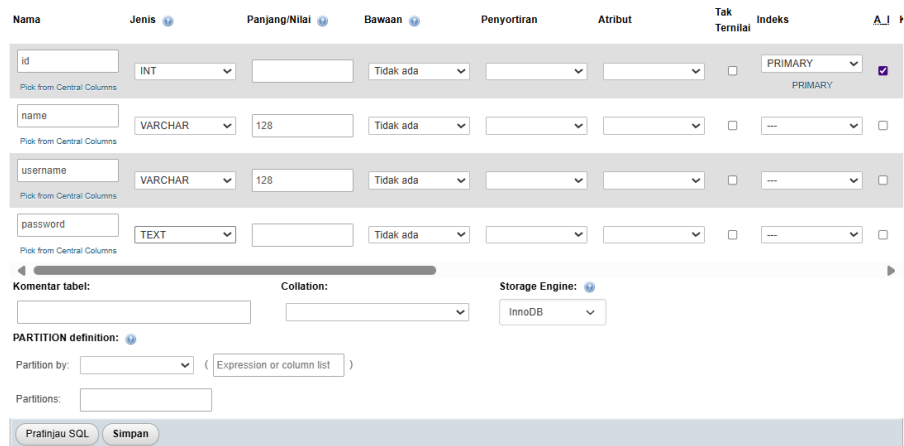
Buat

3. Setelah itu, buat tabel user dengan cara klik database laundry_apps dan buat tabel yang diberi nama user



The screenshot shows a simple form for creating a table. It has two input fields: 'Nama tabel' (Table Name) containing the text 'user' and 'Jumlah kolom' (Number of columns) containing the number '4'. To the right of these fields is a button labeled 'Buat' (Create).

4. Klik creat atau buat, akan muncul tampilannya, dan isi seperti pada gambar dibawah lalu klik simpan



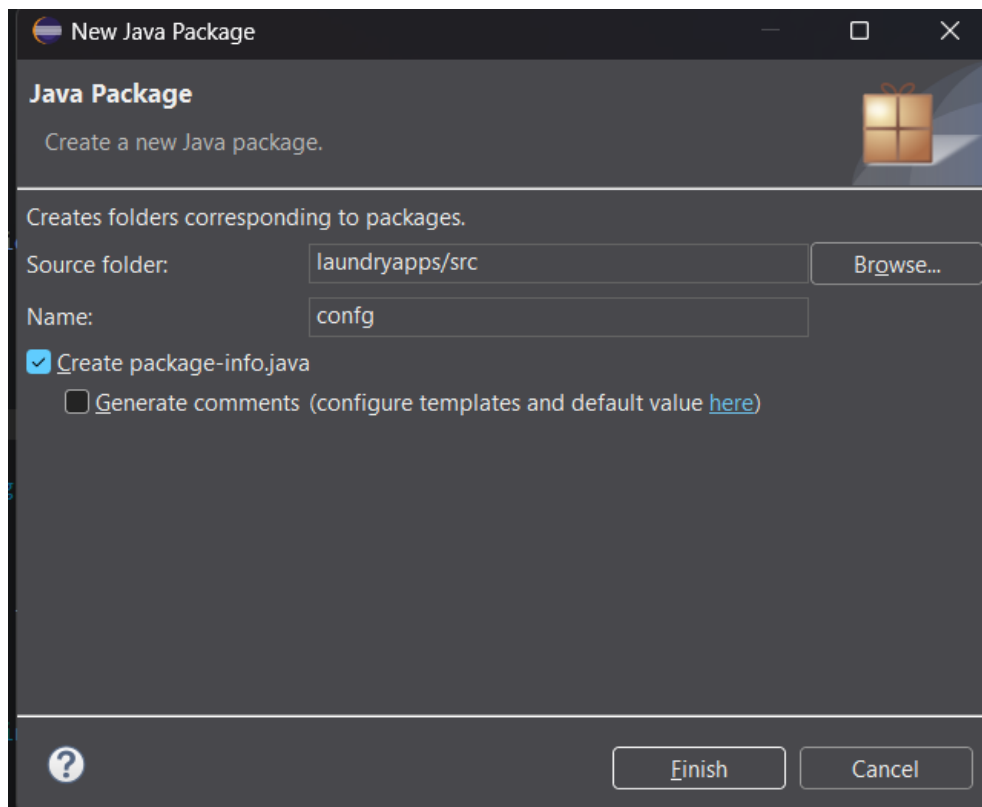
This screenshot shows a more detailed configuration interface for a table named 'user'. It features a table with columns for defining each field:

Nama	Jenis	Panjang/Nilai	Bawaan	Penyortiran	Atribut	Tak Tertilai	Indeks
id	INT		Tidak ada			☐	PRIMARY
name	VARCHAR	128	Tidak ada			☐	
username	VARCHAR	128	Tidak ada			☐	
password	TEXT		Tidak ada			☐	

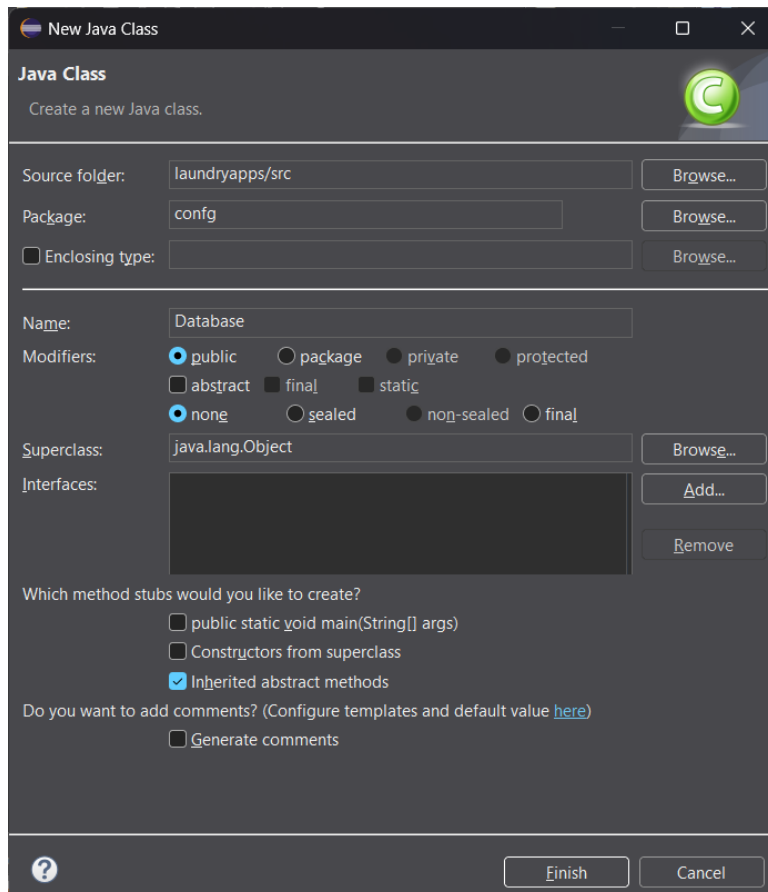
Below the table, there are sections for 'Komentar tabel:' (Table Comment), 'Collation:', 'Storage Engine:' (set to InnoDB), and 'PARTITION definition:'. At the bottom, there are buttons for 'Pratinjau SQL' and 'Simpan' (Save).

D. Membuat Koneksi ke Database MySQL

1. Buat package baru pada src dengan nama config. Package ini digunakan untuk mengatur konfigurasi aplikasi yang akan dibuat, termasuk pengaturan database



2. Setelah itu, bikin class baru pada package config dengan nama Database



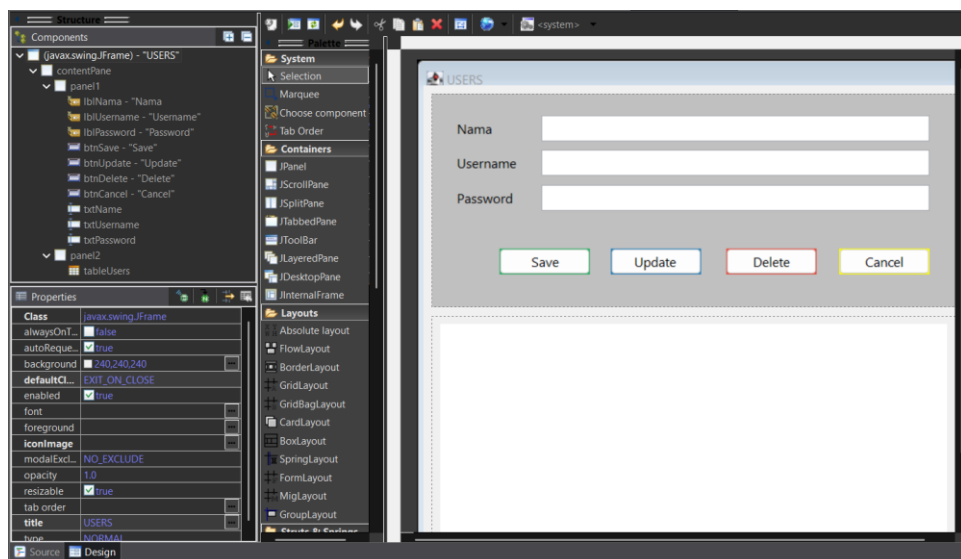
3. Selanjutnya, lakukan konfigurasi sesuai dengan kode program berikut

```
1 package config;
2
3 import java.sql.*;
4 import javax.swing.JOptionPane;
5
6 import com.sun.jdi.connect.spi.Connection;
7
8
9 public class Database {
10     Connection conn;
11
12     public static Connection koneksi() {
13         try {
14             Class.forName("com.mysql.cj.jdbc.Driver");
15             Connection conn = (Connection) DriverManager.getConnection("jdbc:mysql://localhost/laundry_apps", "root", "");
16             return conn;
17         } catch (Exception e) {
18             JOptionPane.showMessageDialog(null, e);
19             return null;
20         }
21     }
22 }
```

Kode ini membuat kelas Java bernama Database yang bertugas menghubungkan aplikasi ke database MySQL. Ada metode koneksi() yang memuat driver MySQL dan mencoba menghubungkan ke database laundry_apps di server lokal dengan username root tanpa password. Jika berhasil, metode ini mengembalikan objek koneksi yang bisa dipakai untuk menjalankan perintah SQL. Jika gagal, program akan menampilkan pesan error lewat jendela pop-up dan mengembalikan null sebagai tanda koneksi tidak berhasil

E. Membuat Tampilan CRUD User

1. Buat file baru bernama UserFrame di package ui menggunakan JFrame, seperti gambar berikut



F. Membuat Tabel Model

1. Buat package baru dengan nama table
2. Setelah itu, buat file baru bernama TableUser di dalam package table, lalu isi dengan kode program berikut

```
1 package table;
2
3 import javax.swing.table.AbstractTableModel;
4
5 import model.User;
6
7 import java.util.List;
8
9 public class TableUser extends AbstractTableModel {
10     List<User> ls;
11     private String[] columnNames = {"ID", "Name", "Username", "Password"};
12
13     public TableUser(List<User> ls) {
14         this.ls = ls;
15     }
16
17     @Override
18     public int getRowCount() {
19         return ls.size();
20     }
21
22     @Override
23     public int getColumnCount() {
24         return 4;
25     }
26
27     @Override
28     public String getColumnName(int column) {
29         return columnNames[column];
30     }
31
32     @Override
33     public Object getValueAt(int rowIndex, int columnIndex) {
34         switch (columnIndex) {
35             case 0:
36                 return ls.get(rowIndex).getId();
37             case 1:
38                 return ls.get(rowIndex).getNama();
39             case 2:
40                 return ls.get(rowIndex).getUsername();
41             case 3:
42                 return ls.get(rowIndex).getPassword();
43             default:
44                 return null;
45         }
46     }
47 }
```

Kode ini membuat kelas TableUser sebagai model tabel khusus untuk JTable di Java Swing. Kelas ini menghubungkan data berupa daftar objek User dengan tampilan tabel. Dengan mewarisi AbstractTableModel, kelas ini mengatur jumlah baris sesuai jumlah user, menentukan ada empat kolom, memberikan nama kolom, dan mengambil data seperti ID, nama, username, dan password untuk ditampilkan di tiap sel tabel. Intinya, TableUser mengubah data User menjadi format tabel yang bisa dilihat di antarmuka

G. Membuat Fungsi DAO

1. Buat package baru dengan nama DAO
2. Setelah itu, buat class interface baru bernama UserDao, lalu isi dengan kode program berikut

```
1 package DAO;  
2  
3 import java.util.List;  
4  
5 import model.User;  
6  
7 public interface UserDao {  
8     void save(User user);  
9     public List<User> show();  
10    public void delete(String id);  
11    public void update(User user);  
12 }
```

Kode ini membuat interface Java bernama UserDao yang mengikuti pola desain Data Access Object (DAO). Interface ini berfungsi sebagai kontrak yang menetapkan operasi standar untuk mengelola data pengguna, seperti menyimpan data baru, menampilkan daftar pengguna, memperbarui data, dan menghapus pengguna berdasarkan ID. Setiap kelas yang mengimplementasikan UserDao harus menyediakan keempat metode tersebut (CRUD)

H. Menggunakan Fungsi DAO

1. Buat class baru bernama UserRepo di package DAO yang akan digunakan untuk mengimplementasikan interface DAO yang sudah dibuat.
2. Terapkan UserDao menggunakan kata kunci implements

```
13 public class UserRepo implements UserDao {
```

3. Membuat instansiasi Connection, mendefinisikan konstruktor, dan menyiapkan String untuk manipulasi database.

```

15     private Connection connection;
16     final String insert = "INSERT INTO user (name, username, password) VALUES (?, ?, ?);";
17     final String select = "SELECT * FROM user;";
18     final String delete = "DELETE FROM user WHERE id=?;";
19     final String update = "UPDATE user SET name=?, username=?, password=? WHERE id=?;";
20
21     public UserRepo() {
22         connection = (Connection) Database.koneksi();
23     }

```

Kode ini adalah bagian awal dari kelas UserRepo yang berfungsi mengelola akses data pengguna. Di dalamnya ada empat perintah SQL untuk operasi dasar: menambah, mengambil, menghapus, dan memperbarui data pengguna. Tanda tanya (?) dipakai sebagai tempat pengganti nilai pada perintah SQL agar lebih aman. Ada juga konstruktor UserRepo() yang membuat koneksi ke database dengan memanggil metode Database.koneksi() dan menyimpannya untuk digunakan saat melakukan operasi data

4. Buat method save, lalu isi dengan kode program berikut

```

25     @Override
26     public void save(User user) {
27         PreparedStatement st = null;
28         try {
29             st = connection.prepareStatement(insert);
30             st.setString(1, user.getNama());
31             st.setString(2, user.getUsername());
32             st.setString(3, user.getPassword());
33             st.executeUpdate();
34
35         } catch (SQLException e) {
36             e.printStackTrace();
37         } finally {
38             try {
39                 st.close();
40             } catch (SQLException e) {
41                 e.printStackTrace();
42             }
43         }
44     }

```

Kode ini adalah implementasi metode save yang berfungsi menyimpan data objek User ke database dengan menggunakan PreparedStatement yang aman. Dia menyiapkan perintah insert, lalu mengganti tanda tanya (?) dengan data nama, username, dan password dari objek user. Setelah itu, perintah untuk menambah data dijalankan lewat executeUpdate(). Semua proses ini ditangani dalam blok try-catch untuk mengatasi error, serta blok finally memastikan PreparedStatement selalu ditutup agar sumber daya tidak bocor

5. Membuat method show yang berfungsi untuk mengambil data dari database

```

46     @Override
47     public List<User> show() {
48         List<User> ls = null;
49         try {
50             ls = new ArrayList<User>();
51             Statement st = connection.createStatement();
52             ResultSet rs = st.executeQuery(select);
53             while(rs.next()) {
54                 User user = new User();
55                 user.setId(rs.getString("id"));
56                 user.setName(rs.getString("name"));
57                 user.setUsername(rs.getString("username"));
58                 user.setPassword(rs.getString("password"));
59                 ls.add(user);
60             }
61         } catch (SQLException e) {
62             Logger.getLogger(UserDao.class.getName()).log(Level.SEVERE, null, e);
63         }
64         return ls;
65     }
66 }

```

Kode ini adalah metode show yang mengambil semua data pengguna dari database dan mengembalikannya dalam bentuk List objek User. Prosesnya dimulai dengan menjalankan perintah SQL SELECT * FROM user, lalu hasilnya disimpan dalam ResultSet. Kode akan membaca setiap baris data, membuat objek User baru dengan data tersebut, dan menambahkannya ke dalam ArrayList. Jika ada error, akan tercatat menggunakan Logger. Setelah semuanya selesai, metode ini mengembalikan daftar pengguna yang sudah lengkap.

6. Membuat method update yang berfungsi untuk mengubah data

```

68     @Override
69     public void update(User user) {
70         PreparedStatement st = null;
71         try {
72             st = connection.prepareStatement(update);
73             st.setString(1, user.getName());
74             st.setString(2, user.getUsername());
75             st.setString(3, user.getPassword());
76             st.setString(4, user.getId());
77             st.executeUpdate();
78         } catch (SQLException e) {
79             e.printStackTrace();
80         } finally {
81             try {
82                 st.close();
83             } catch (SQLException e) {
84                 e.printStackTrace();
85             }
86         }
87     }

```

Kode ini adalah metode update yang mengubah data pengguna di database. Metode ini menerima objek User berisi data terbaru. Dengan menggunakan PreparedStatement, data nama, username, dan password diisi ke placeholder pertama hingga ketiga, lalu ID pengguna diisi pada placeholder keempat untuk menentukan baris yang akan diubah. Setelah itu, executeUpdate() dijalankan

untuk menerapkan perubahan. Blok try-catch menangani error, dan finally memastikan PreparedStatement selalu ditutup agar sumber daya tidak bocor

7. Membuat method delete yang berfungsi untuk menghapus data

```
89      @Override
90      public void delete(String id) {
91          PreparedStatement st = null;
92          try {
93              st = connection.prepareStatement(delete);
94              st.setString(1, id);
95              st.executeUpdate();
96          } catch (SQLException e) {
97              e.printStackTrace();
98          } finally {
99              try {
100                  st.close();
101              } catch (SQLException e) {
102                  e.printStackTrace();
103              }
104          }
105      }
106  }
```

Kode ini adalah metode delete yang menghapus data pengguna dari database berdasarkan ID. Metode ini menerima parameter id yang digunakan untuk menentukan data mana yang dihapus. Menggunakan PreparedStatement, id dimasukkan ke dalam perintah SQL delete dengan aman. Setelah itu, executeUpdate() dijalankan untuk menghapus data. Blok try-catch-finally digunakan untuk menangani error dan memastikan PreparedStatement selalu ditutup agar sumber daya tidak terbuang

I. Menggunakan Fungsi CRUD DAO pada GUI

1. Buat method reset pada JFrame seperti kode berikut untuk menghapus nilai inputan setelah proses berhasil dilakukan

```
public void reset() {
    txtName.setText("");
    txtUsername.setText("");
    txtPassword.setText("");
}
```

Kode ini membuat metode reset yang berfungsi menghapus isi dari beberapa kolom input di antarmuka pengguna. Ketika dijalankan, metode ini

mengosongkan teks pada txtName, txtUsername, dan txtPassword. Biasanya metode ini dipakai untuk membersihkan form setelah data berhasil disimpan atau saat pengguna menekan tombol reset.

2. Membuat objek instance dari UserFrame

```
UserRepo usr = new UserRepo();  
List<User> ls;  
public String id;
```

Kode ini mendeklarasikan tiga variabel penting untuk mengelola data di antarmuka pengguna. Pertama, objek UserRepo bernama usr untuk mengakses database. Kedua, sebuah List bernama ls untuk menyimpan data pengguna yang diambil dari database. Ketiga, variabel id yang menyimpan ID pengguna yang dipilih agar mudah digunakan untuk update atau hapus data.

D. Kesimpulan

Dari praktikum yang telah dilakukan, kita berhasil menghubungkan aplikasi Java dengan database MySQL menggunakan MySQL Connector, sehingga memungkinkan pengelolaan database secara efisien baik melalui antarmuka pengguna maupun langsung dari database. Dengan kemampuan ini, mahasiswa diharapkan dapat membuat tabel user di MySQL, menghubungkan Java dengan database, merancang tampilan GUI untuk operasi CRUD, serta membuat dan mengimplementasikan interface dalam pemrograman. Hal ini menunjukkan bahwa integrasi antara Java dan MySQL dapat mendukung pembuatan aplikasi berbasis database yang interaktif dan fungsional